In [5]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

tips = px.data.tips()
iris = px.data.iris()
stocks = px.data.stocks()

tips.head()
```

Out[5]:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

Task 1: Draw a line chart of monthly sales

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"] sales = [12, 15, 14, 20, 24, 22] # in thousands

- Draw a line chart of sales against months using plt.plot() with circle markers and a grid.
- Add a title, an x-axis label and a y-axis label.
- Which month had the highest sales?

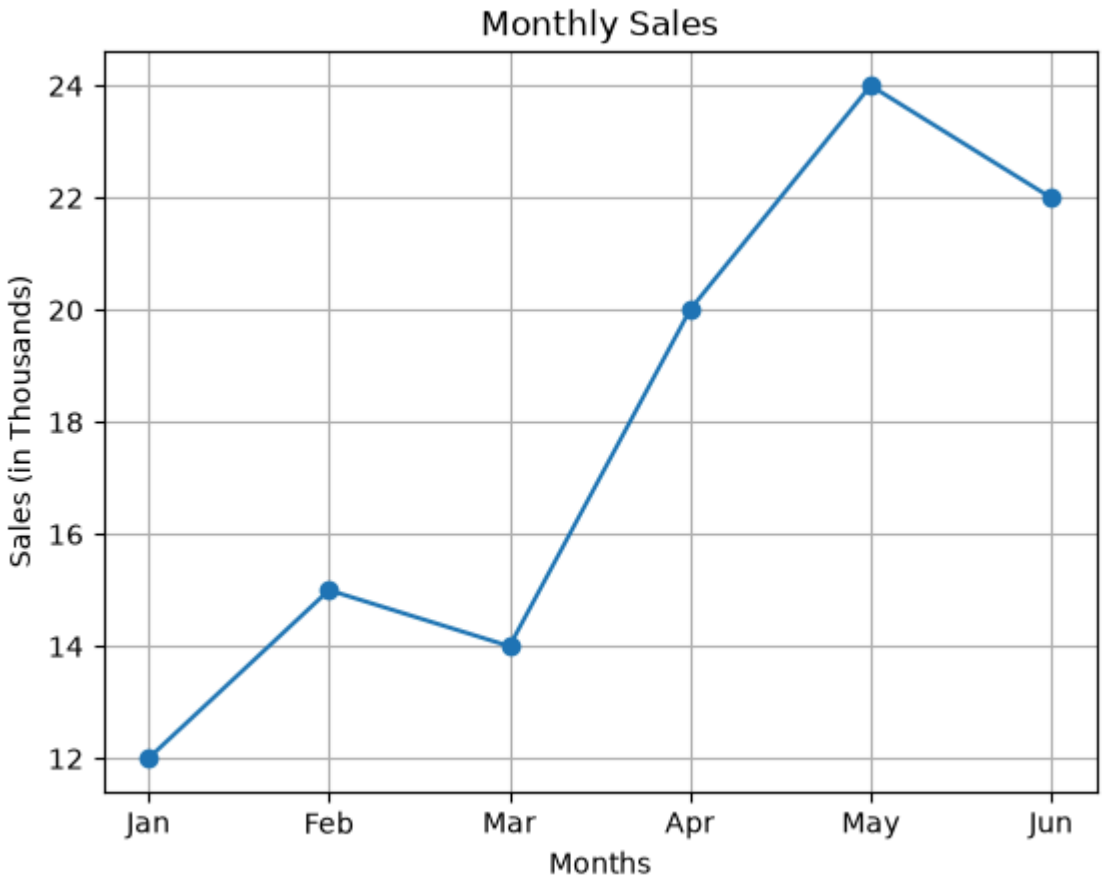
In [6]:
```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [12, 15, 14, 20, 24, 22]

plt.plot(months, sales, marker='o')
plt.title("Monthly Sales")
plt.xlabel("Months")
plt.ylabel("Sales (in Thousands)")
plt.grid(True)

plt.show()

# Find the month with highest sales
max_sales = max(sales)
highest_month = months[sales.index(max_sales)]

print("Highest Sales Month:", highest_month)
print("Sales:", max_sales, "thousand")
```




```
Highest Sales Month: May
Sales: 24 thousand
```

Task 2: Find average tip by day and draw a bar chart

- Find the average tip for each day: avg_tip = tips.groupby("day")["tip"].mean().reindex(["Thur", "Fri", "Sat", "Sun"])
- Draw a bar chart using plt.bar() with a colour of your choice.
- On which day are tips the highest?

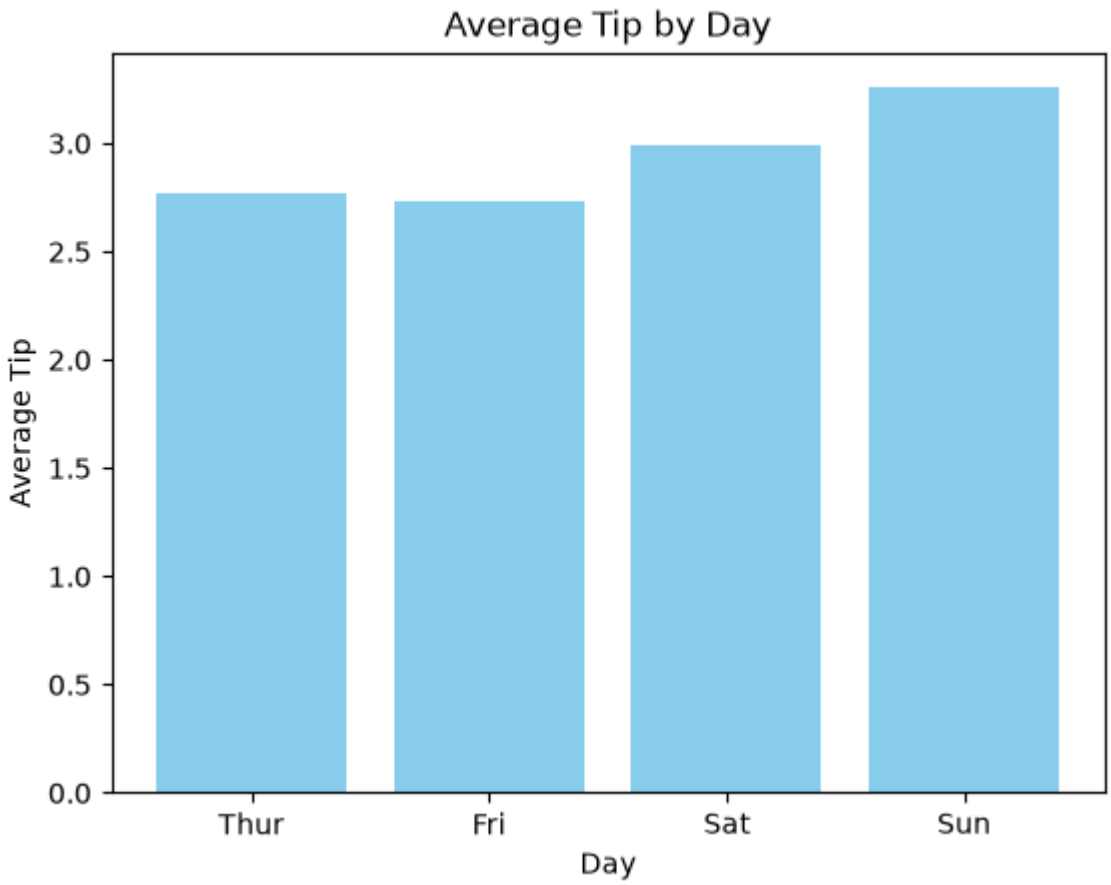
In [7]:
```
avg_tip = tips.groupby("day")["tip"].mean().reindex(["Thur", "Fri", "Sat", "Sun"

plt.bar(avg_tip.index, avg_tip.values, color="skyblue")
plt.title("Average Tip by Day")
plt.xlabel("Day")
plt.ylabel("Average Tip")

plt.show()

# Find the day with the highest average tip
highest_day = avg_tip.idxmax()
highest_tip = avg_tip.max()

print("Highest Tip Day:", highest_day)
print("Average Tip:", round(highest_tip, 2))
```




```
Highest Tip Day: Sun
Average Tip: 3.26
```

Task 3: Histogram and Scatter Plot Side by Side

- Create two charts in one figure using fig, axes = plt.subplots(1, 2, figsize=(12, 4)).
- On the left, draw a histogram of total_bill with 10 bins using axes[0].hist().
- On the right, draw a scatter plot of total_bill (x) against tip (y) using axes[1].scatter().
- Give each chart a title and axis labels using set_title(), set_xlabel() and set_ylabel().
- Save the figure with plt.savefig("tips_plots.png", dpi=150).
- In 2 lines, describe the most common bill range and the relationship between bill and ti

In [9]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

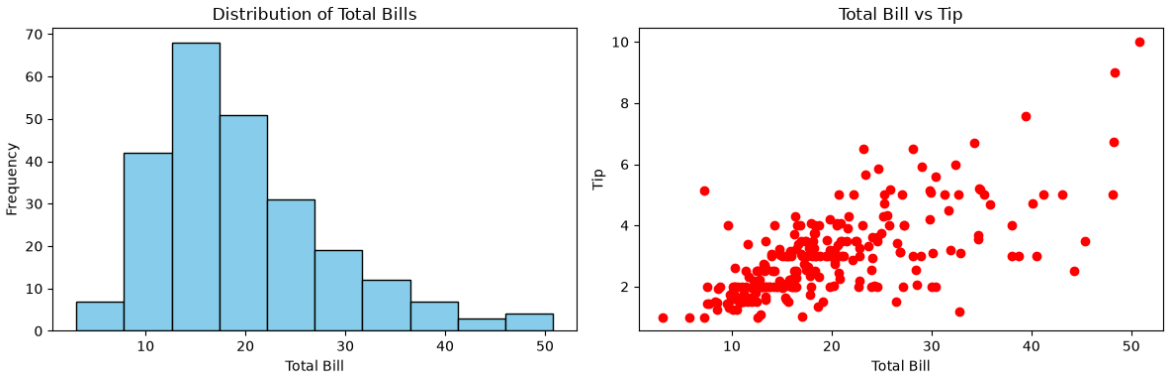
# Histogram
```

```
axes[0].hist(tips["total_bill"], bins=10, color="skyblue", edgecolor="black")
axes[0].set_title("Distribution of Total Bills")
axes[0].set_xlabel("Total Bill")
axes[0].set_ylabel("Frequency")

# Scatter Plot
axes[1].scatter(tips["total_bill"], tips["tip"], color="red")
axes[1].set_title("Total Bill vs Tip")
axes[1].set_xlabel("Total Bill")
axes[1].set_ylabel("Tip")

plt.tight_layout()
plt.savefig("tips_plots.png", dpi=150)

plt.show()
```



- **Description (2 lines):**

The most common total bill range is approximately 10 to 20, as shown by the histogram.

The scatter plot shows a positive relationship between total bill and tip, meaning customers generally leave larger tips for larger bills

Task 4: Count Plot and Bar Plot

- Draw sns.countplot(data=tips, x="day") to show the number of customers on each day.
- Draw sns.barplot(data=tips, x="day", y="total_bill") to show the average bill on each day.
- Which day has the most customers? Which day has the highest average bill?

In [15]:
```
plt.figure(figsize=(12, 5))

# Count Plot
plt.subplot(1, 2, 1)
sns.countplot(data=tips, x="day")
plt.title("Number of Customers by Day")
plt.xlabel("Day")
```

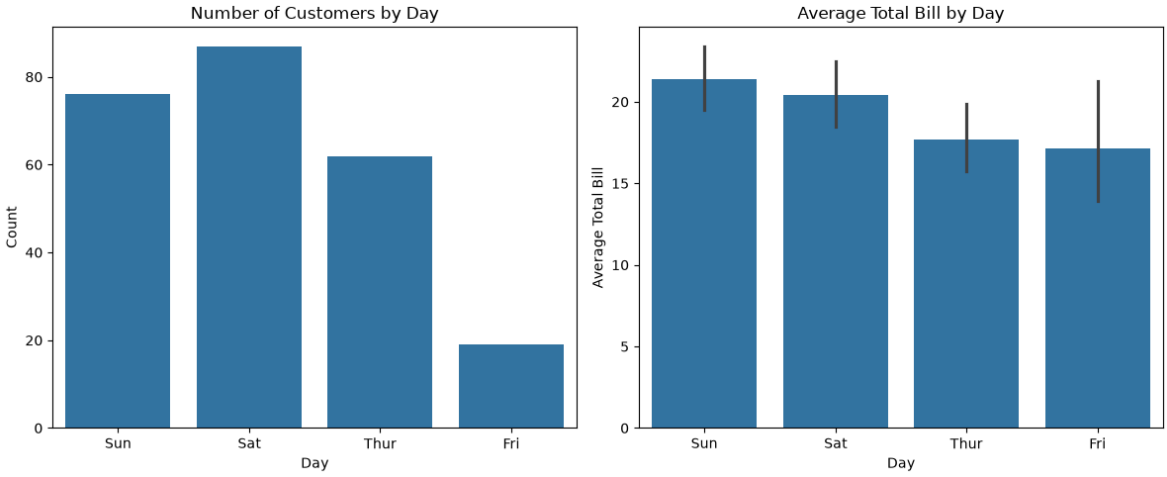
```
plt.ylabel("Count")

# Bar Plot
plt.subplot(1, 2, 2)
sns.barplot(data=tips, x="day", y="total_bill")
plt.title("Average Total Bill by Day")
plt.xlabel("Day")
plt.ylabel("Average Total Bill")

plt.tight_layout()
plt.show()

# Find answers
most_customers = tips["day"].value_counts().idxmax()
highest_avg_bill = tips.groupby("day")["total_bill"].mean().idxmax()

print("Day with most customers:", most_customers)
print("Day with highest average bill:", highest_avg_bill)
```

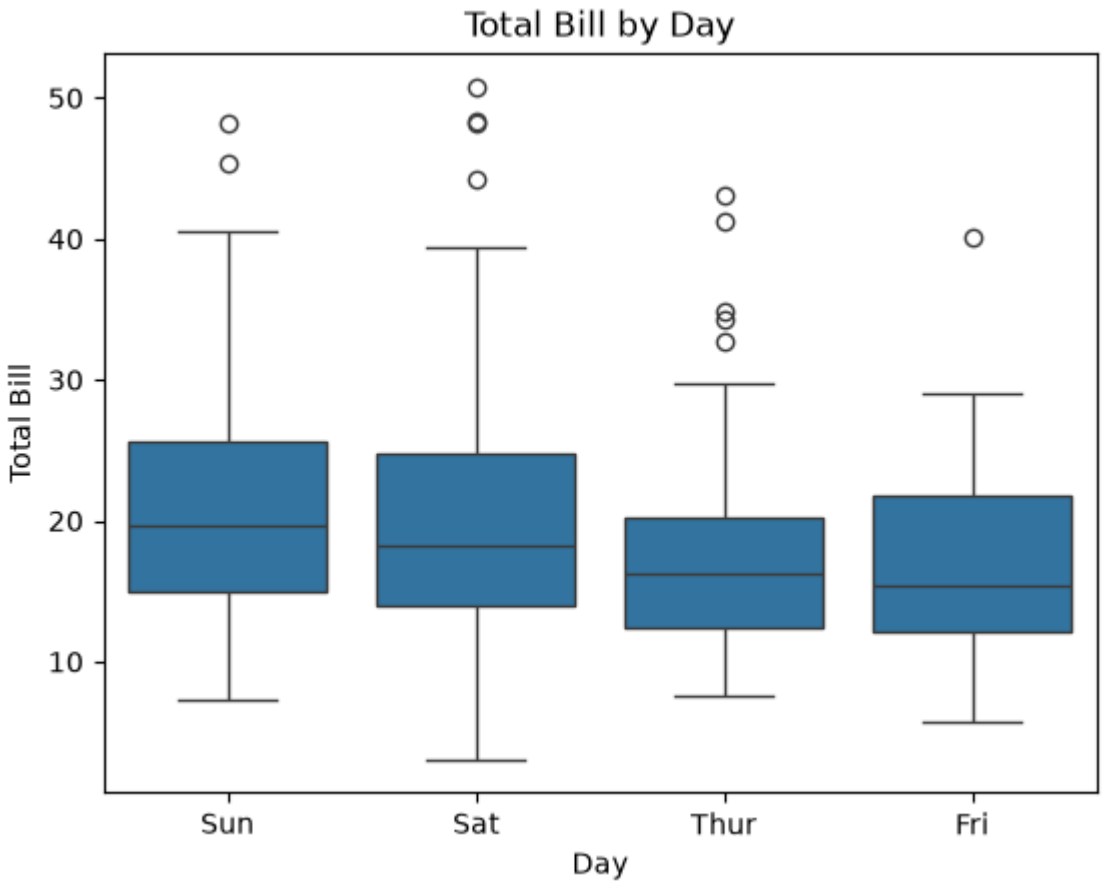



```
Day with most customers: Sat
Day with highest average bill: Sun
```

Task 5: Box Plot

- Draw sns.boxplot(data=tips, x="day", y="total_bill").
- Which day has the highest median bill? Are there dots beyond the whiskers? What do they represent?
- Redraw the plot with hue="sex" added and note one difference you see.

In [16]:
```
sns.boxplot(data=tips, x="day", y="total_bill")
plt.title("Total Bill by Day")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```

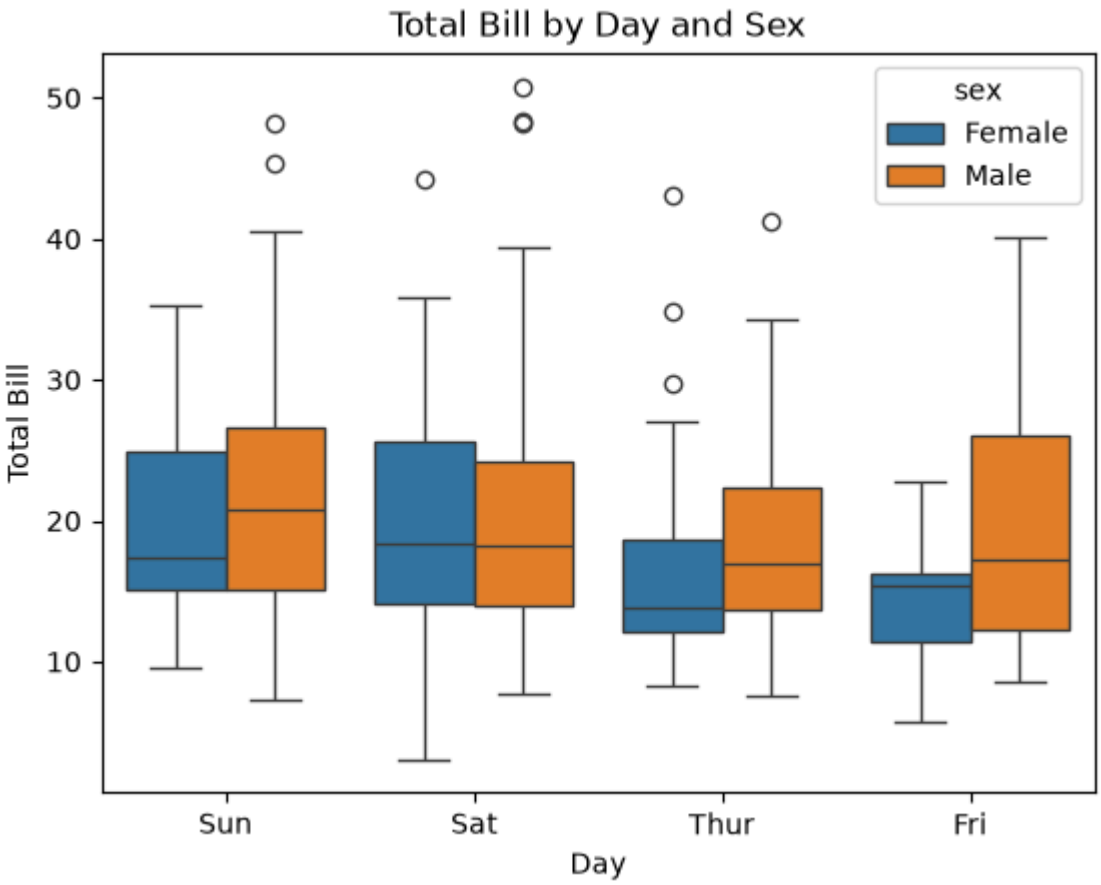


In [17]:
```
median_bill = tips.groupby("day")["total_bill"].median()
print(median_bill)
```

```
day
Fri     15.38
Sat     18.24
Sun     19.63
Thur    16.20
Name: total_bill, dtype: float64
```

- The day with the highest median bill is Sun.
- Yes, there are often dots beyond the whiskers. These represent outliers, which are unusually high or low bill amounts compared to the rest of the data.

In [18]:
```
sns.boxplot(data=tips, x="day", y="total_bill", hue="sex")
plt.title("Total Bill by Day and Sex")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```



- > One Difference

  The plot separates bills for Male and Female customers on each day, making it easier to compare their bill distributions. In many cases, male customers show a wider spread of bill amounts.

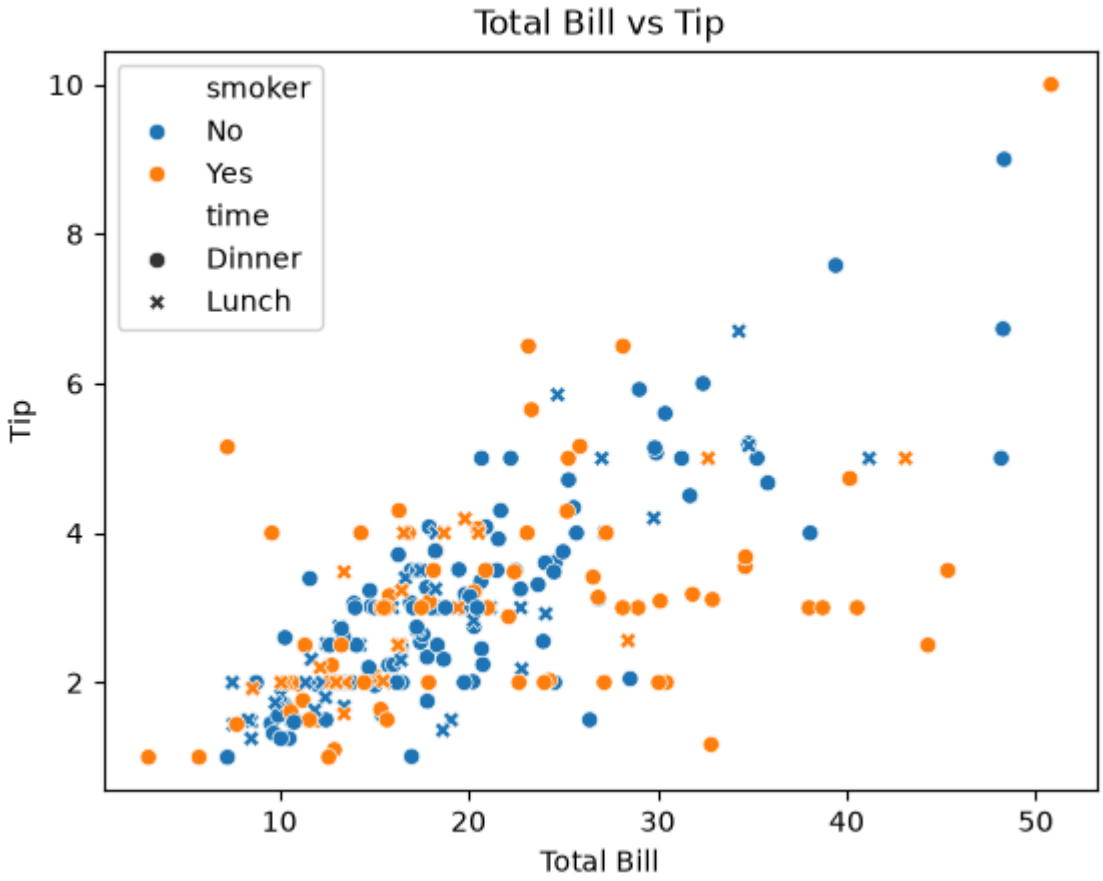
Task 6: Scatter Plot with Colour and Style

- > Draw sns.scatterplot(data=tips, x="total_bill", y="tip", hue="smoker", style="time").
- > In one line, say whether smokers and non-smokers tip differently

In [19]:
```
sns.scatterplot(
    data=tips,
    x="total_bill",
    y="tip",
    hue="smoker",
    style="time"
)

plt.title("Total Bill vs Tip")
plt.xlabel("Total Bill")
plt.ylabel("Tip")
```

```
plt.show()
```



- **Observation (1 line):**

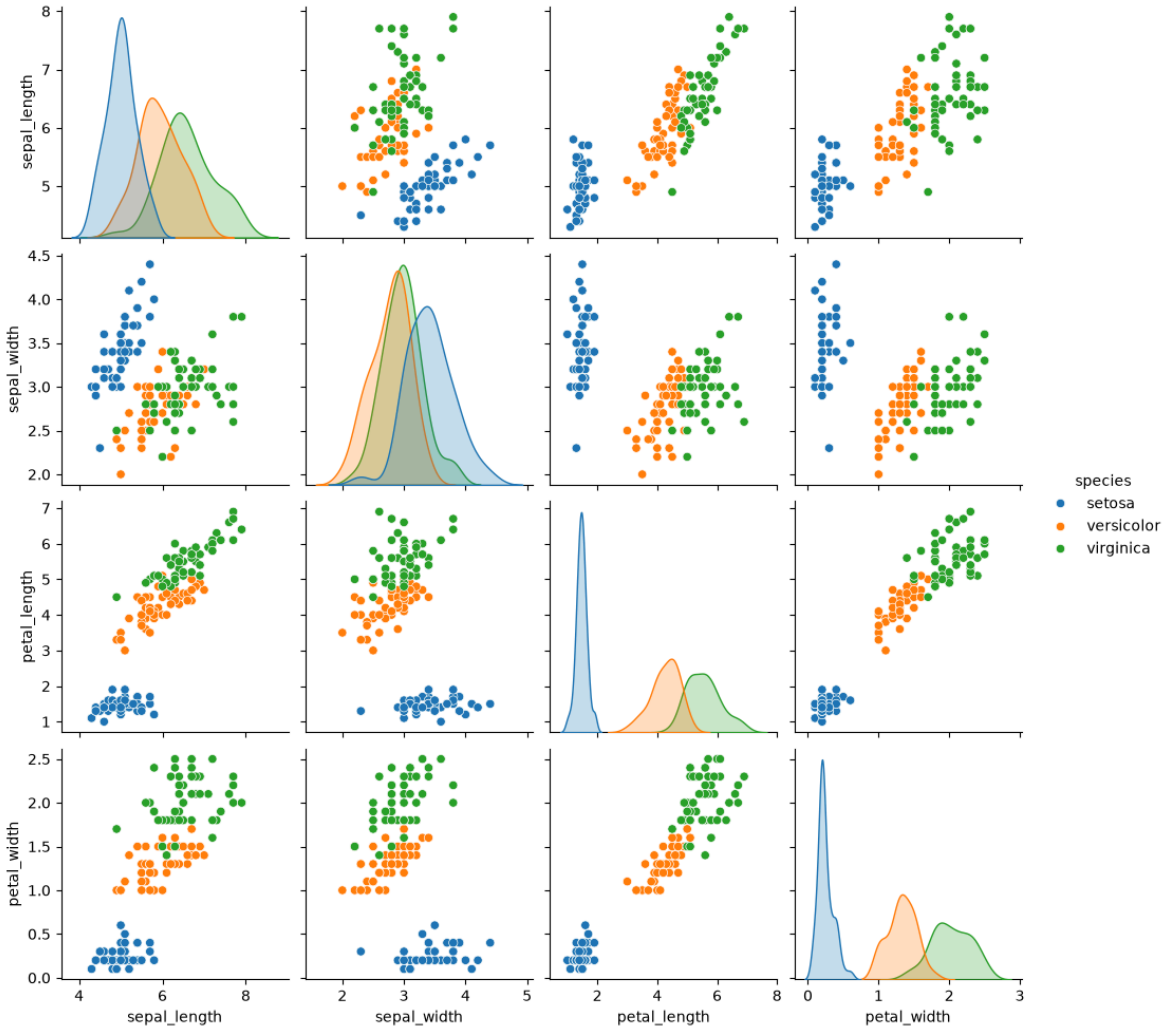
Smokers and non-smokers show a similar tipping pattern, with tips generally increasing as the total bill increases.

Task 7: Pair Plot

- Draw sns.pairplot(iris, hue="species", vars=["sepal_length", "sepal_width", "petal_length", "petal_width"]).
- Which two measurements separate the three species best? Which species is the easiest to identify?

In [20]:
```
sns.pairplot(
    iris,
    hue="species",
    vars=["sepal_length", "sepal_width", "petal_length", "petal_width"]
)

plt.show()
```



Petal length and petal width separate the three species the best. Setosa is the easiest species to identify because its petal measurements are clearly separated from the other two species.

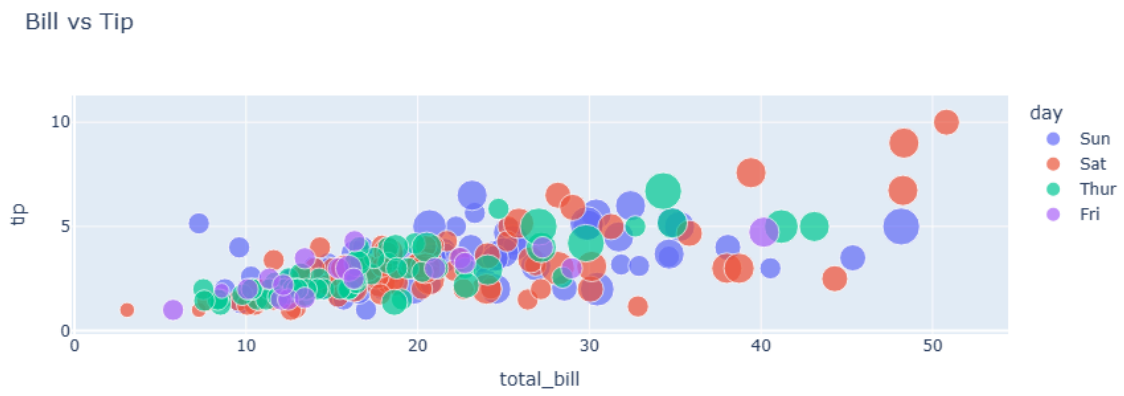
Task 8: Interactive Scatter Plot

- > Create the chart and display it with fig.show(): fig = px.scatter(tips, x="total_bill", y="tip", color="day", size="size", hover_data=["sex", "time"], title="Bill vs Tip") fig.show()
- > Hover over points, zoom into an area, and click a day in the legend to hide it.
- > Write two things you can do in this chart that you could not do in your Matplotlib charts.

In [21]:
```
fig = px.scatter(
tips,
x="total_bill",
y="tip",
color="day",
size="size",
```

```
hover_data=["sex", "time"],
title="Bill vs Tip"
)

fig.show()
```



- You can hover over points to see detailed information such as sex, time, total bill, and tip.
- You can zoom, pan, and hide/show categories by clicking the legend, making the chart interactive and easier to explore

Task 9: Interactive Box Plot

- Draw px.box(tips, x="day", y="total_bill", color="sex").
- Draw px.histogram(tips, x="total_bill", color="time", nbins=15).
- Which meal, Lunch or Dinner, has larger bills in general?

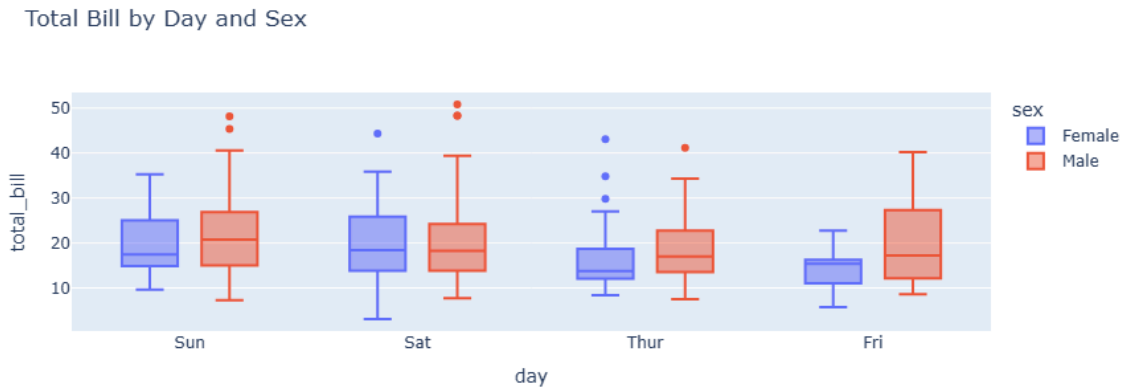
In [23]:
```
fig = px.box(
    tips,
    x="day",
    y="total_bill",
    color="sex",
    title="Total Bill by Day and Sex"
)

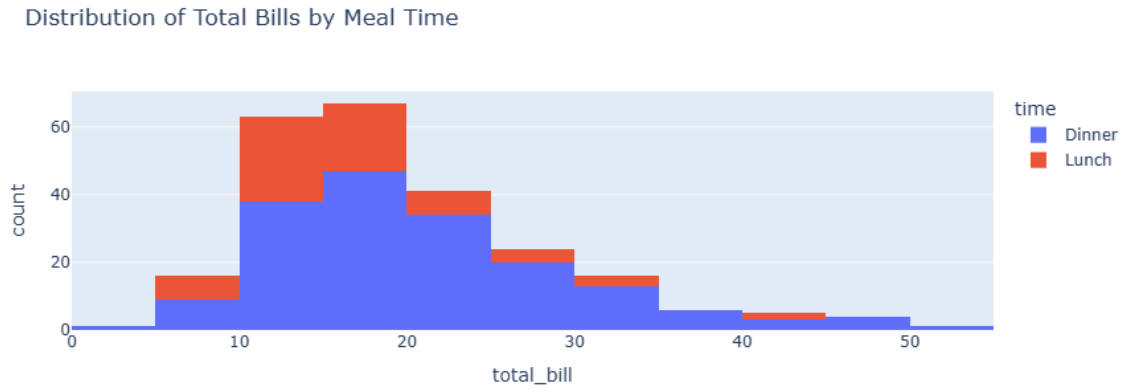
fig.show()

# Interactive Histogram

fig = px.histogram(
    tips,
    x="total_bill",
    color="time",
    nbins=15,
    title="Distribution of Total Bills by Meal Time"
)
```

```
fig.show()
```





- > Dinner generally has larger bills than Lunch, as the dinner bills are spread over higher total bill values and have a higher overall range

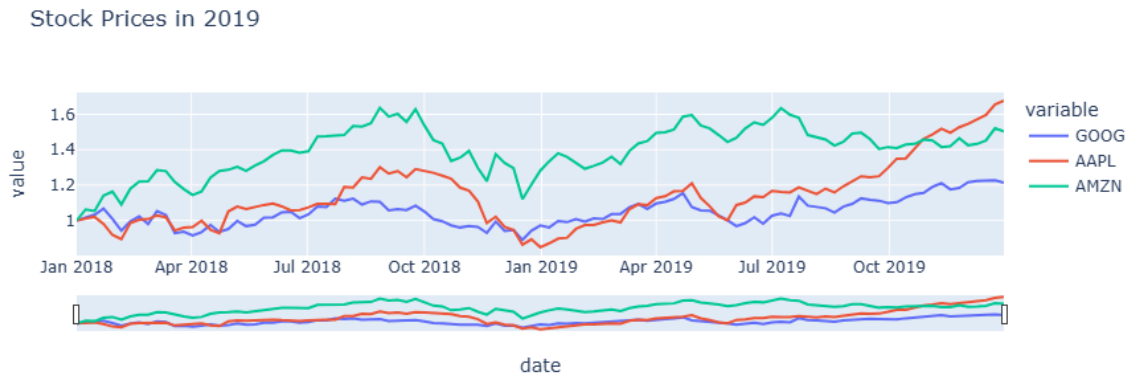
Task 10: Line Chart with Range Slider

- > Draw px.line(stocks, x="date", y=["GOOG", "AAPL", "AMZN"]), then add fig.update_xaxes(rangeslider_visible=True).
- > Which of the three stocks grew the most by the end of 2019?
- > Use the slider to zoom into any three-month period and note what happened there.

In [24]:
```
fig = px.line(
    stocks,
    x="date",
    y=["GOOG", "AAPL", "AMZN"],
    title="Stock Prices in 2019"
)

fig.update_xaxes(rangeslider_visible=True)
```

```
fig.show()
```



- AAPL (Apple) showed the greatest growth by the end of 2019.
- Using the range slider to zoom into a three-month period (for example, October to December), you can observe that all three stocks generally increased, with AAPL showing the strongest upward trend

Task 11:

Feature	Matplotlib	Seaborn	Plotly
Ease of use	Medium	Easy	Easy
Interactive charts?	No	No	Yes
Best used for	Basic customizable charts	Statistical attractive charts	Interactive web charts

- For a lab report, I would choose Matplotlib or Seaborn because they produce clear, static, and publication-ready charts that can be easily saved in reports.
- For a web page, I would choose Plotly because it provides interactive features such as zooming, hovering, and filtering, which make data exploration easier for users.

In [ ]: